

Route Guards

A guard is a function the router runs **before** opening a page. It decides whether the user may enter, and where to send them if not. This guide covers **canActivate**, **canActivateChild**, **canDeactivate**, **canMatch** and **resolve**, how to redirect correctly, and how to test a guard.

1 · WHY GUARDS EXIST

Routing says *which* page; a guard says *whether*

Without a guard, anyone who types `/admin` into the address bar gets the admin page. A **route guard** is a small function the router calls **before** it activates a route. Its answer decides what happens next.

User clicks a link → Router finds the route → Guard runs → Page opens

If the guard says no, the router can **cancel** the navigation (the user stays where they were) or **redirect** to another URL such as a login page.

TYPICAL USE	GUARD QUESTION
Login required	Is the user signed in?
Roles / permissions	May this user see the admin area?
Unsaved changes	Really leave this form with unsaved edits?
Feature flags	Does this route even exist for this user?
Data first	Load what the page needs before it shows (resolver)

IMPORTANT

A guard only controls what the **browser** shows. Anyone can change client-side code, so **never** treat a guard as real security. The server must check the token on every API call.

2 · THE GUARD IN ONE FUNCTION

canActivate: return true, false, or a UrlTree

Modern Angular guards are plain **functions** (`CanActivateFn`). They run inside an injection context, so `inject()` works for services and the router.

```
export const membersGuard: CanActivateFn = () => {
  const auth = inject(AuthService);
  const router = inject(Router);

  return auth.loggedIn()
    ? true // let the user in
    : router.createUrlTree(['login'], { // redirect instead
        queryParams: { returnUrl: '/members' },
      });
};

// app.routes.ts
{ path: 'members', canActivate: [membersGuard], loadComponent: () => import('./members') }
```

2 · THE GUARD IN ONE FUNCTION · CONTINUED

RETURN VALUE	WHAT THE ROUTER DOES
<code>true</code>	Continues: the page opens.
<code>false</code>	Cancels the navigation. The user stays put (and sees nothing happen).
<code>UrlTree</code>	Cancels this navigation and redirects to that URL as part of the same navigation.
<code>Observable</code> / <code>Promise</code>	Any of the above, delivered later (e.g. after an HTTP check). An Observable must emit and complete (use <code>take(1)</code> or <code>first()</code>).

3 · REDIRECT THE RIGHT WAY

Return a `UrlTree` — don't call `router.navigate()` inside the guard

Calling `router.navigate()` from a guard starts a **second** navigation while the first is still in progress. Returning a `UrlTree` lets the router cancel the first one and redirect as one clean step, so history and the back button behave.

Avoid

```
if (!auth.loggedIn()) {
  router.navigate(['/login']);
  return false;
}
return true;
```

Prefer

```
return auth.loggedIn()
  ? true
  : router.createUrlTree(['/login']);
```

TIP

Angular 18.2+ also has `RedirectCommand` (`new RedirectCommand(urlTree, { skipLocationChange: true })`) when you need extra navigation options on the redirect.

4 · THE FIVE KINDS

Pick the guard that matches your question

GUARD	QUESTION IT ANSWERS	TYPICAL CODE
<code>canActivate</code>	May the user open this route?	login / role check
<code>canActivateChild</code>	Same, for every child under a parent route	one guard on an <code>/admin</code> parent
<code>canDeactivate</code>	May the user leave this page?	unsaved-changes prompt
<code>canMatch</code>	Should this route match at all ? A failing route is skipped and the router tries the next one; it also stops a lazy chunk from being downloaded.	role-based routes, feature flags
<code>resolve</code>	Not a yes/no: fetch the data the page needs before it shows.	load a product by id
<code>canLoad</code> still exists but is deprecated — use <code>canMatch</code> .		

5 · MORE EXAMPLES

canDeactivate, canMatch and resolve

```
// canDeactivate: ask before leaving a dirty form
export interface HasUnsavedChanges { hasChanges(): boolean }
export const leaveGuard: CanDeactivateFn<HasUnsavedChanges> = (page) =>
  !page.hasChanges() || confirm('Discard your changes?');

// canMatch: this route only exists for admins (and its chunk is never downloaded otherwise)
export const adminMatch: CanMatchFn = () => inject(AuthService).isAdmin();
{ path: 'admin', canMatch: [adminMatch], loadChildren: () => import('./admin/routes') }

// resolve: load data before the page appears
export const productResolver: ResolveFn<Product> = (route) =>
  inject(ProductService).getById(Number(route.paramMap.get('id')));
{ path: 'products/:id', resolve: { product: productResolver }, component: ProductPage }
```

6 · ORDER OF EVENTS

What happens during a navigation

- 1 `canMatch` runs while the router works out which route matches.
- 2 `canDeactivate` runs on the page being **left**.
- 3 `canActivateChild` (parent → child), then `canActivate` run on the page being **opened**.
- 4 `resolve` loads the data the page needs.
- 5 The route activates and the component is created.

Several guards in one array must **all** allow the navigation. If one returns a `UrlTree`, that redirect is used instead.

7 · TESTING A GUARD

It is just a function — run it in an injection context

```
it('redirects to /login while logged out', () => {
  TestBed.configureTestingModule({ providers: [provideRouter([])] });

  const result = TestBed.runInInjectionContext(() => membersGuard({} as any, {} as any));

  expect(result instanceof UrlTree).toBeTrue();
  expect(TestBed.inject(Router).serializeUrl(result as UrlTree)).toBe('/login');
});
```

Replace `AuthService` with a fake in `providers` to test both the allowed and the blocked path without any real login.

8 · COMMON MISTAKES

Things that trip people up

- **Trusting the guard for security.** It only hides pages in the browser. Always verify on the server.
- **Calling `router.navigate()` in the guard.** Return a `UrlTree` instead.
- **An Observable that never completes.** The navigation waits forever. Add `take(1)`.
- **Redirect loops.** A guard that redirects to a route guarded by itself. Send users to a public page (e.g. `/login`).
- **Using a class-based guard.** `CanActivate` classes are deprecated; use functional guards.
- **Forgetting `canMatch` for lazy `loadChildren` routes.** `canActivate` runs after those child routes were already downloaded; `canMatch` runs first and prevents the download.

9 · CHEAT SHEET & QUICK QUIZ

I WANT TO...	USE
Block a page unless logged in	<code>canActivate</code> returning a <code>UrlTree</code> to /login
Protect all children of a parent	<code>canActivateChild</code> on the parent
Warn about unsaved changes	<code>canDeactivate</code>
Hide a route (and its lazy code) by role	<code>canMatch</code>
Load data before the page shows	<code>resolve</code>

Q: Which return value redirects?

A `UrlTree`. `false` only cancels.

Q: Why is a guard not enough for security?

Client code can be changed. The server must check every request.

Q: Which guard stops a lazy chunk from downloading?

`canMatch` (`canLoad` is deprecated).

Try it live: youssefezzat17.github.io/angular-lab/guards — flip the login switch and open the members page.